

Response to Reviewers

Previous submission: Computer Networks, COMNET-D-25-04940R2

Post-Quantum Authentication Latency in TLS 1.3: A Validated Model and the Limits of Deferred Signature Verification

(previously: *Asynchronous Post-Quantum Signature Verification for Low-Latency TLS 1.3 Handshakes*)

We thank Reviewers #4 and #5 for their careful reading. Their comments led to a substantial rewrite and to an entirely new evaluation, and to a change of focus that we believe makes the paper considerably stronger. We have redone the evaluation with the setup the reviewers asked for: three-certificate hybrid chains, X25519MLKEM768 key exchange, client and server in separate network namespaces on disjoint CPU cores joined by an emulated 1500-byte-MTU link, more than 46,000 individually timed handshakes, and a measured bounded worker pool. We also added experiments on the TCP initial window, congestion control, packet loss and verifier speed. The new results do not reproduce the latency and throughput gains reported in the previous version; instead they show precisely when deferral helps, and that network round trips dominate PQ authentication latency. We have reframed the paper around a validated latency model accordingly (new title). The main changes are:

1. New evaluation on OpenSSL 3.5 (Sections 6 and 7) and a new, parameter-free latency model (Section 5) that predicts the round-trip count of all 68 tested configurations and the median latency within 2.2% on average. For lattice-based hybrids on an AVX2 CPU, deferral does not measurably reduce latency and does not increase throughput; its gain follows the verification work removed minus a dispatch cost ($R^2 = 0.999$), so it helps SLH-DSA (13.5%) and clients without vector instructions (9–26%). Δt is 0.08 ms at the median with a bounded pool. The earlier 14–21 ms savings are withdrawn.
2. The contribution is repositioned as a client-side, implementation-level scheduling strategy with specified local state semantics, not a protocol mechanism (R5.1).
3. The motivation now separates confidentiality (HNDL, addressed by hybrid key exchange) from authentication (PKI migration lead time, Mosca’s inequality), and explains what DAPV does and does not address (R4 detailed 1).
4. New Background (PQ schemes, hybrid constructions, TLS 1.3 authentication modes) and a broadened Related Work section with 15 new references (R4.1, R4.2, R4 detailed 2).
5. The latency model is corrected: extra round trips arise from the TCP initial congestion window, not from exceeding the MTU. The new data confirm the model’s predictions for all five schemes (0, 0, 0, 1 and 2 extra round trips).
6. The exponential risk model (old Eq. 6, Fig. 11) and the mixed-baseline efficiency metric (old Eq. 7, Table 3) are removed; the leakage bound now includes the congestion-window limit and uses measured Δt (R5.3, R5.4).
7. No interpolated, extrapolated or projected values remain (R5.8, R5.9). Each result opens with its takeaway (R5.6, R5.7).
8. Implementation, drivers and raw data are released (R4.3).

Reviewer #4

Comment. W1. *Writing style: introduction and background/related work are too short; decisions such as using threads rather than instruction-level parallelism (AVX2/AVX-512) are not justified.*

Response. The Introduction has been rewritten and extended (Sections 1.1–1.5), and the former one-page “Background and Related Work” is now two sections (Sections 2 and 3). The threads-versus-SIMD question is answered in the new Section 4.6: the two are complementary. SIMD shortens one

verification and is already used (liboqs AVX2 code); deferral moves the remaining time off the critical path. We also acknowledge explicitly that for one AVX2 ML-DSA-44 verification (≈ 0.1 ms) deferral can save at most that amount, and identify the cases where synchronous PQ work is large enough to matter: multi-certificate chains, multi-algorithm credentials, SLH-DSA and devices without vector units. Independent verifications in these cases can be parallelised across cores, which SIMD cannot do.

Changes. Sections 1, 2, 3 and 4.6.

Comment. *W2. The literature review is too narrow.*

Response. Section 3 now covers PQ-TLS measurement studies, size-reduction work (KEMTLS and its variants, intermediate-certificate suppression, mixed certificate chains), hybrid-signature theory and IETF composite signatures, asynchronous cryptographic offload in TLS (QTLS/OpenSSL async), TLS False Start and 0-RTT as precedents for accepting a bounded weaker guarantee, and formal analyses of deferred authentication. Table 9 positions DAPV relative to these by the cost addressed and approach. In revising the bibliography we also found and corrected two entries that did not match the cited works (the former Cremers2016 and Cheval2022).

Changes. Section 3, Table 6, bibliography.

Comment. *W3. The implementation was not made available.*

Response. The provider patch, client, server, link emulator, experiment drivers, raw per-handshake and per-job measurements, and analysis scripts are released at **[AUTHOR INPUT: URL and DOI]**. Section 6.1 describes the implementation in detail.

Changes. Sections 6.1 and 6.2.

Comment. *W4. Methodology: client and server on the same machine compete for CPU; number of repetitions not justified and too few; why multithread if PQC verification is fast; how many certificates are used; throughput with one host is hard to trust.*

Response. We agree with every point and have redone the evaluation. (i) *Co-location*: client and server now run in separate network namespaces with their own TCP/IP stacks, pinned to disjoint cores, with traffic crossing an emulated link rather than loopback (Section 6.3). We also measured the co-located placement explicitly. In neither placement does DAPV increase the completed-handshake rate (Table 6); the earlier throughput gains are withdrawn. The remaining limitation, that both namespaces share one virtual machine, is stated in Section 7.7. (ii) *Repetitions*: 1000 handshakes per configuration at zero RTT and 200 at each other RTT, interleaved in ten rounds; 43,200 individually timed PQ verifications; throughput runs repeated three times. Differences are reported with bootstrap confidence intervals. (iii) *Why threads*: Section 4.6 discusses this, and the new data answer it empirically. For ML-DSA and FN-DSA (37–59 μ s per verification) threads bring no benefit, and a thread per job is slower. Deferral helps only for expensive verification (SLH-DSA-128f, 0.57 ms). (iv) *Certificates*: all experiments use three-certificate chains (root, intermediate, leaf; the server sends two), so each handshake performs three PQ verifications, all of which DAPV defers. A single-certificate configuration is kept for comparison.

Changes. Sections 5–7 (new), Tables 2–8, Figures 2–8.

Comment. *W5. netem may not trigger real TCP behaviour, local traffic bypasses the NIC queue, a single certificate does not trigger TCP mechanisms, and it is unclear whether PQC key exchange is used.*

Response. All addressed. Traffic no longer uses loopback: each namespace has a TAP interface with a 1500-byte MTU and segmentation and receive offloads disabled, so TCP sends real 1500-byte frames and runs its own congestion control. A userspace link emulator adds the delay (Section 6.3). With three-certificate chains, the TCP mechanisms the reviewer anticipated are now visible. P-384 + ML-DSA-65 needs one extra round trip and SLH-DSA-128f two, exactly as predicted from the initial congestion window (Tables 2 and 3, Section 5). X25519MLKEM768 is used in all runs, so both confidentiality and authentication are post-quantum; X25519 is measured for comparison (+0.19 ms for the hybrid KEM). Beyond delay, the emulator now also drops frames: at 3% loss, the fraction of delayed handshakes grows with the number of segments of the first flight, as a simple model predicts (Section 7.6, Table 7). We also vary the server’s initial congestion window and the congestion controller (BBR, CUBIC), which the reviewer’s remark on TCP mechanisms prompted: a larger initial window removes the extra round trips of large PQ chains.

Changes. Sections 5, 6.3, 6.4, 7.1 and 7.6.

Comment. *W6. Marginal contribution; consider hybrids with more than two algorithms, which could use multiple verification threads.*

Response. We agree that a multithreaded verifier alone was a marginal contribution, and the new data confirm it. The paper’s contribution is now (1) a parameter-free latency model of PQ authentication in TLS 1.3, validated on 68 configurations, which shows that congestion-window round trips, not computation, dominate; (2) a precise break-even analysis of deferred verification (gain = $0.97W - 0.14\text{ms}$, $R^2 = 0.999$) that tells implementers exactly when it is worthwhile; (3) the effect of initial window, congestion control and loss; and (4) a formal verification of the provisional state in Tamarin, which proves that promoted sessions keep the full hybrid guarantees and finds attacks on unsafe variants (session tickets used before promotion; PQ signature not bound to the transcript), establishing which design rules are necessary (Section 8.1, Table 8). Three-certificate chains already give three independent verifications per handshake, which DAPV spreads over two workers. Algorithm 1 generalises directly to multi-algorithm credentials, which we list as future work together with devices without vector units.

Comment. *D1. The motivation is HNDL, which concerns confidentiality, while the proposal concerns authentication; PKI migration is not mentioned; the hybrid-for-interoperability claim is inaccurate; why hybrid, why concatenation, what alternatives; fragmentation is motivated but not addressed.*

Response. All accepted. Section 1.1 separates HNDL (confidentiality, countered by hybrid key exchange) from the authentication threat (active impersonation once a CRQC exists) and explains why authentication must nevertheless migrate now: PKI lead times and Mosca’s inequality. Section 1.2 corrects the statement about interoperability: hybrid authentication is a hedge against breaks of the young PQ schemes, not an interoperability mechanism; a composite certificate is not verifiable by clients that do not support it. Section 2.2 describes the four hybrid constructions (composite/concatenation, nested extensions, parallel chains, mixed chains), their trade-offs, and which of them DAPV applies to. The abstract and introduction now state that DAPV addresses only the computational component of the cost and not size or fragmentation.

Changes. Abstract, Sections 1.1, 1.2, 2.2.

Comment. *D2. Background does not explain PQC, available algorithms, how to choose, what hybrids are, or how TLS authentication works (not only certificates; 0-RTT, PSK; Certificate message also carries signatures).*

Response. New Section 2.1 describes ML-DSA, FN-DSA and SLH-DSA, the assumptions behind them and the selection trade-offs, with a size table (Table 1). Section 2.3 describes the three TLS 1.3 authentication modes (certificate-based, PSK/resumption with 0-RTT, post-handshake), notes that chain validation also requires signature verifications in the `Certificate` message, and explains how DAPV interacts with each mode. A new rule (Section 4.3) prevents a provisional session’s tickets from being used for resumption or 0-RTT before promotion. Section 2.3 now states that the implementation defers the PQ components of the chain signatures as well as of `CertificateVerify`.

Comment. *D3. Section 3 needs more discussion; some messages of Fig. 1 are not explained; design goals.*

Response. Fig. 1 has been redrawn and now shows all TLS 1.3 messages; every numbered step is explained in Section 4.2. Design goals and non-goals are listed in Section 4.1.

Comment. *D4. The key-exchange algorithm is not stated; ML-KEM adds bytes and computation.*

Response. X25519MLKEM768 is now used in all experiments (Section 6.4), and X25519 is measured for comparison. Table 1 gives the X25519MLKEM768 share sizes (1216/1120 bytes). The reviewer’s figure of 800 bytes corresponds to the ML-KEM-512 public key.

Comment. *D5. Algorithm 1 dispatches the PQC verification after verifying ECDSA. Why not dispatch first to run in parallel? Algorithms are not explained.*

Response. Explained in Section 4.5 (“Dispatch order”): dispatching only after σ_E verifies prevents an attacker without a valid classical signature from consuming workers, and the classical verification takes only tens of microseconds, so the difference in Δt is negligible. Implementations may dispatch first and cancel on failure. All three algorithms are now explained in the text, and the functions they use are defined.

Comment. *Minor: “carries” should be “carry”.*

Response. The sentence has been rewritten.

Reviewer #5

Comment. *1. Is DAPV a protocol-level contribution or an implementation-level scheduling optimisation?*

Response. It is implementation-level; we agree with the reviewer. DAPV changes no message and needs no server support. All “protocol-level” claims have been removed; the Introduction (Section 1.4) states the positioning explicitly, and the standardisation discussion (Section 9) now says that no TLS extension is required, only implementation guidance on the provisional semantics.

Comment. *2. What happens in the transition from provisional to fully_authenticated? Is there re-keying?*

Response. No. Section 4.3 (“Promotion does not re-key”) explains that the transition is a local state change, why re-keying is unnecessary (the keys depend only on the key-exchange secret and the transcript), and what this means for data sent during Δt in both the success and failure cases.

Comment. *3. Justify Equation (6): where does β come from, and what event does $R(\Delta t, B)$ represent?*

Response. On re-examination we could not justify β empirically, and the exponential form did not correspond to a well-defined event. We have removed Eq. (6) and its figure. Exposure is now characterised only by the deterministic bound of Eq. (3), which we have strengthened to include the TCP congestion-window limit.

Comment. 4. *Justify Equation (7), which mixes baselines.*

Response. We agree the mixed baselines were not meaningful. The metric and its table have been removed; the results report latency and rate changes directly against a single, consistent baseline: synchronous verification of the same configuration (Tables 3, 4 and 6).

Comment. 5. *Justify the 8 KB cap.*

Response. It is a heuristic default, not an empirical result. Section 8.2 (“Choice of D_{policy} ”) explains that the correct value depends on what data the application will reveal to a classical-only authenticated peer, and gives the reasoning behind 8 KB as a default.

Comment. 6. *Clarify the purpose of Tables 2 and 4.*

Response. Old Table 4 has been removed because most of its entries were interpolated or extrapolated (see comment 8). Every table in the new evaluation is introduced by the conclusion it supports: Table 2 (the latency model predicts every round-trip count), Table 3 (deferral does not help lattice schemes on AVX2 hardware but helps SLH-DSA), Table 4 (it helps them without vector instructions), Table 5 (the provisional window is a fraction of a millisecond), Table 6 (deferral does not raise throughput) and Table 7 (loss penalises larger flights).

Comment. 7. *Improve the presentation of the evaluation.*

Response. The evaluation section has been rewritten around the new experiments and a predictive model (Section 5). Each subsection of Section 7 opens with a one-sentence takeaway, and all eleven previous evaluation figures and the old efficiency and interpolated tables have been removed.

Comment. 8. *Why were interpolation and extrapolation used?*

Response. They should not have been. All values in the revised paper are measured, at all five RTTs, for every configuration (Table 3 and Figure 3).

Comment. 9. *Was the $\Delta t < 1$ ms thread-pool result implemented or projected?*

Response. It was a projection in the previous version. We have now implemented and measured a bounded pool. Δt is 0.079 ms at the median and 0.42 ms at the 99th percentile for ML-DSA-44, versus 0.114 and 0.99 ms with a thread per job (Table 5, Figure 5).

Comment. 10. *VerifyPQC() is not defined; unresolved references in Section 5.7.*

Response. VerifyPQC and VerifyECDSA are defined in Section 4.5, including the message they verify. The unresolved references (to an undefined equation label and figure label) have been fixed; the revised manuscript compiles without undefined references.

We thank both reviewers again.